

CAP Theorem

Complete Study Notes

Consistency · Availability · Partition Tolerance
CP Systems · AP Systems · Trade-offs · Distributed Design

TOPICS COVERED

1. What is the CAP Theorem?

2. Consistency (C) explained

3. Availability (A) explained

4. Partition Tolerance (P) explained

4. CP vs AP — Behaviour During Partition

5. CA — When is it valid?

6. Quick Revision & Interview Questions

7. Real-world Systems (MongoDB, Cassandra)

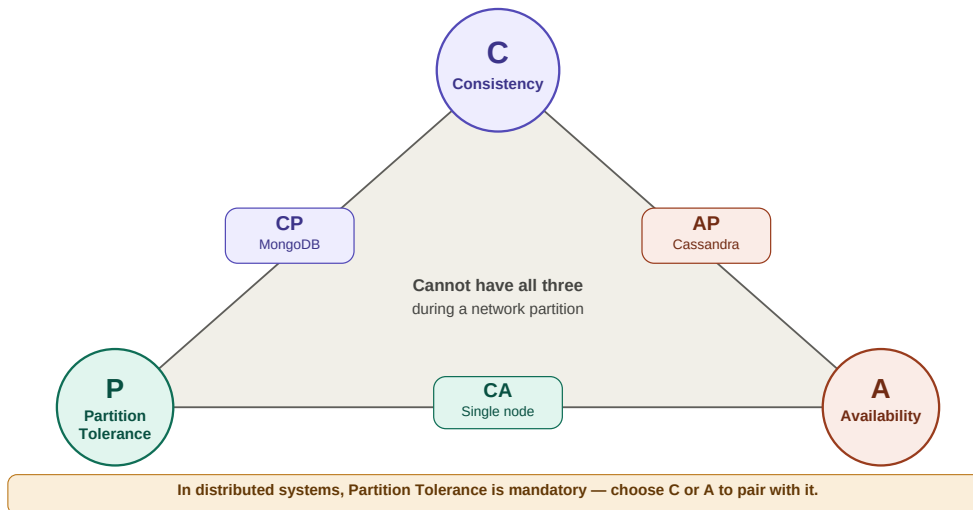
Table of Contents

1. What is the CAP Theorem?	3
2. The Three Properties	3
2.1 Consistency (C)	3
2.2 Availability (A)	3
2.3 Partition Tolerance (P)	3
3. Why Not All Three?	4
4. CP vs AP vs CA — System Types	4
5. Behaviour During a Network Partition	4
6. Real-world Systems	5
7. Quick Revision & Interview Questions	5

1. What is the CAP Theorem?

The **CAP theorem** states that a distributed system can guarantee at most **two of three properties** simultaneously: Consistency, Availability, and Partition Tolerance. It was formulated by Eric Brewer and proven by Gilbert and Lynch in 2002.

CAP Theorem — Pick Any Two



CAP triangle — each edge represents a system type; all three vertices cannot be held at once during a partition

2. The Three Properties

2.1 Consistency (C)

Every read returns the **most recent write**. All nodes see the same data at the same time. A write to node A must propagate to all other nodes before any read can return.

- If node A is updated, node B must reflect that update before responding to reads
- Achieved by blocking reads on out-of-sync nodes during replication
- Example: Bank balance — a debit must be visible everywhere immediately

2.2 Availability (A)

Every request receives a **response** — success or failure. The system is always operational. Nodes may return stale data if consistency is not prioritised.

- Every node responds to every request, regardless of other nodes' state
- May serve outdated data if a network partition has prevented sync
- Example: Social media feed — showing a slightly old post count is acceptable

2.3 Partition Tolerance (P)

The system **continues operating despite network partitions** — when nodes cannot communicate with each other due to a network failure.

- A partition means node A cannot reach node B (e.g., India ↔ USA link breaks)
- P is mandatory in any real distributed system — network failures always happen

- Since P is required, the real choice is always between C and A during a partition

Partition Tolerance is non-negotiable in geo-distributed systems. You cannot prevent network failures between datacenters.

The real design decision: when a partition occurs, do you prioritise Consistency (CP) or Availability (AP)?

3. Why Not All Three Simultaneously?

When a **network partition** occurs, nodes cannot communicate. The system faces an impossible choice:

If you want Consistency	If you want Availability
• Block reads/writes on out-of-sync nodes • Some nodes become unavailable • Availability is sacrificed → CP system	• Let all nodes respond independently • Nodes may return stale data • Consistency is sacrificed → AP system

C+A+P together is impossible during a partition. You must drop one. Since P is mandatory, the choice is always CP or AP.

4. CP vs AP vs CA — System Types

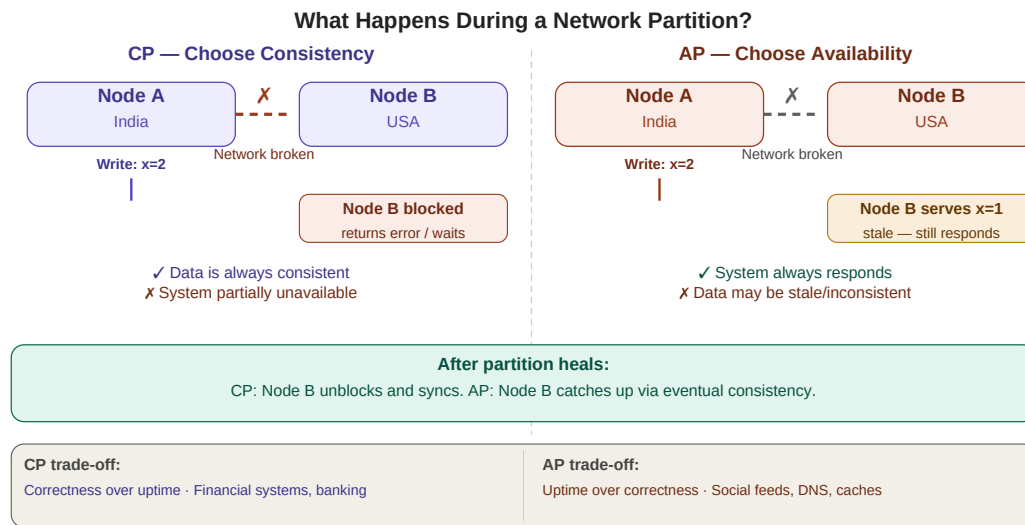
CAP System Types — Comparison

Type	Guarantees	Sacrifices	Real-world use cases
CP	Consistency + Partition Tolerance	Availability (blocks during partition)	Banking, payments, distributed locks
AP	Availability + Partition Tolerance	Consistency (may serve stale data)	Social feeds, DNS, shopping carts
CA	Consistency + Availability	Sacrifices partition tolerance	Single-node RDBMS, not truly distributed

CA is theoretical in distributed systems — a network partition will always be possible across nodes.

CAP system types — guarantees, sacrifices, and real-world examples

5. Behaviour During a Network Partition



CP blocks the partitioned node to stay consistent; AP lets all nodes respond with potentially stale data

6. Real-world Systems

System	Type	Behaviour
MongoDB (default)	CP	Blocks reads from secondaries during partition to ensure latest data
Apache Cassandra	AP	Always responds; uses tunable consistency levels; eventual consistency
Apache ZooKeeper	CP	Leader election and coordination; blocks if leader unreachable
Amazon DynamoDB	AP	High availability; eventual consistency by default; strong optional
MySQL / PostgreSQL	CA	Single node; consistent + available but no partition tolerance
Apache Kafka	CP	Durable, ordered log; leader must be reachable for writes
DNS	AP	Always responds; propagation delay means stale data is common
Banking systems	CP	Correctness over uptime; transactions block rather than serve stale

7. Quick Revision & Interview Questions

Quick Revision

- CAP theorem: a distributed system guarantees at most 2 of 3 — Consistency, Availability, Partition Tolerance.
- Consistency = every read gets the latest write. All nodes see the same data.
- Availability = every request gets a response (may be stale).
- Partition Tolerance = system works even when nodes cannot communicate.
- P is mandatory in distributed systems — network failures always happen.
- The real choice is always CP or AP — you cannot avoid partition tolerance.
- CP: blocks unavailable nodes to stay consistent (banking, Zookeeper, Kafka).

- AP: all nodes respond, possibly with stale data (DNS, Cassandra, social feeds).
- CA: only valid on single-node systems — no partition tolerance possible.
- After a partition heals: CP unblocks and syncs; AP catches up via eventual consistency.

Interview Questions

- 1. What is the CAP theorem? Why can't all three properties coexist?
- 2. What is the difference between Consistency and Availability in CAP?
- 3. Why is Partition Tolerance mandatory in distributed systems?
- 4. What happens to a CP system during a network partition?
- 5. What happens to an AP system during a network partition?
- 6. Give examples of CP systems and AP systems from real-world databases.
- 7. Is CA a valid choice for a distributed system? Why or why not?
- 8. How does eventual consistency relate to the CAP theorem?
- 9. How would you design a payment system under CAP constraints?
- 10. How does MongoDB handle the CAP trade-off by default?

Key Terms

CAP Theorem	Max 2 of 3 — Consistency, Availability, Partition Tolerance
Consistency	Every read returns the most recent write across all nodes
Availability	Every request receives a response (may be stale)
Partition Tolerance	System operates despite network failures between nodes
Network Partition	Communication breakdown between nodes (e.g. datacenter link failure)
CP System	Sacrifices availability to stay consistent during partitions
AP System	Sacrifices consistency to stay available during partitions
CA System	Consistent + available but cannot tolerate partitions — single node only
Eventual Consistency	AP systems sync data after partition heals — not immediately consistent